

[[voltar](#)] [[índice](#)] [[avancar](#)]

14. Algumas funções úteis

O MySQL possui algumas funções que podem ser muito úteis no processamento dos dados. Abaixo podemos ver uma tabela com algumas destas funções:

FUNÇÃO	DESCRIÇÃO
AVG(coluna)	retorna a média dos valores da coluna
COUNT(item)	se <i>item</i> for uma coluna, será retornado o número de valores não NULL nesta coluna. Se a palavra-chave DISTINCT for colocada na frente do nome da coluna, será retornado o número de valores distintos nesta coluna. Se for passado COUNT(*), será retornado o número total de registros independente de quantos tenham valor NULL.
MIN(coluna)	retorna o valor mínimo da coluna
MAX(coluna)	retorna o valor máximo da coluna
SDT(coluna)	retorna o desvio padrão dos valores da coluna
SDTDEV (coluna)	o mesmo que SDT(coluna)
SUM(coluna)	retorna a soma dos valores da coluna

Exemplos da utilização destas funções:

```
mysql> select * from pedido;
+----+-----+-----+
| nr | cliente | valor |
+----+-----+-----+
| 1 | 2 | 50.50 |
| 2 | 2 | 60.00 |
| 3 | 1 | 100.05 |
| 4 | 3 | 54.70 |
| 5 | 3 | 80.90 |
+----+-----+-----+
5 rows in set (0.00 sec)

mysql> select avg(valor) from pedido;
+-----+
| avg(valor) |
+-----+
| 69.230001 |
+-----+
1 row in set (0.00 sec)

mysql>
```

Podemos usar a cláusula `group by` para calcular a média de pedidos para cada elemento de um grupo determinado. Exemplo:

```
mysql> select cliente, avg(valor) from pedido group by cliente;
+-----+-----+
| cliente | avg(valor) |
+-----+-----+
| 1 | 100.050003 |
| 2 | 55.250000 |
| 3 | 67.800001 |
+-----+-----+
3 rows in set (0.04 sec)
```

```
mysql>
```

Observe que quando usamos `group by` a função `avg()` retorna a média calculada para cada componente do grupo especificado, no caso "cliente".

Podemos também testar o valor retornado por `avg()`:

```
mysql> select cliente,avg(valor) from pedido group by cliente having avg(valor) > 60;
+-----+-----+
| cliente | avg(valor) |
+-----+-----+
|      1 | 100.050003 |
|      3 |  67.800001 |
+-----+-----+
2 rows in set (0.02 sec)
```

```
mysql>
```

Note que a cláusula `having` usada aqui tem o mesmo significado que `where` nas consultas normais.

Abaixo vemos exemplos do uso de `count()`:

```
mysql> select pedido.nr,cliente.nome,pedido.valor from
-> cliente left join pedido on cliente.codigo=pedido.cliente;
+-----+-----+-----+
| nr    | nome    | valor  |
+-----+-----+-----+
|      3 | João    | 100.05 |
|      1 | Maria   |  50.50 |
|      2 | Maria   |  60.00 |
|      4 | José    |  54.70 |
|      5 | José    |  80.90 |
| NULL   | Manuel  |  NULL  |
| NULL   | Adão    |  NULL  |
| NULL   | Rodrigo |  NULL  |
| NULL   | Davi    |  NULL  |
| NULL   | Karla   |  NULL  |
| NULL   | Samuel  |  NULL  |
| NULL   | Ana     |  NULL  |
+-----+-----+-----+
12 rows in set (0.00 sec)
```

```
mysql> select count(valor) from cliente left join pedido
-> on cliente.codigo=pedido.cliente;
+-----+
| count(valor) |
+-----+
|           5 |
+-----+
1 row in set (0.00 sec)
```

```
mysql> select count(*) from cliente left join pedido on cliente.codigo=pedido.cliente;
+-----+
| count(*) |
+-----+
|        12 |
+-----+
1 row in set (0.00 sec)
```

```
mysql>
```

Usando `min()`, `max()` e `sum()`:

```
mysql> select * from pedido;
+-----+-----+-----+
```

```
| nr | cliente | valor |
+----+-----+-----+
|  1 |         |  50.50 |
|  2 |         |  60.00 |
|  3 |         | 100.05 |
|  4 |         |  54.70 |
|  5 |         |  80.90 |
+----+-----+-----+
5 rows in set (0.00 sec)
```

```
mysql> select min(valor) from pedido;
+-----+
| min(valor) |
+-----+
|      50.50 |
+-----+
1 row in set (0.00 sec)
```

```
mysql> select max(valor) from pedido;
+-----+
| max(valor) |
+-----+
|     100.05 |
+-----+
1 row in set (0.00 sec)
```

```
mysql> select sum(valor) from pedido;
+-----+
| sum(valor) |
+-----+
|     346.15 |
+-----+
1 row in set (0.00 sec)
```

```
mysql>
```

[[voltar](#)][[índice](#)][[avancar](#)]